VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Belagavi-590014, Karnataka



A dissertation work submitted towards

In partial fulfilment of the requirement for the award of the degree of

Bachelor of Engineering In
Artificial Intelligence and Machine Learning

Submitted by

ANAND PATIL (3GN23AI003)

Under the guidance of

Dr. JASMINEET KAUR AURORA







DEPARTMENT OF ARTIFICIAL INTELLIGENCE G MACHIINE LEARNING
Accredited by NBA, New Delhi

GURU NANAK DEV ENGINEERING COLLEGE BIDAR MAILOOR ROAD, BIDAR-585403, KARNATAKA

Academic Year: 2025-26

Phone: 08482- 226949 **Website: www.gndecb.ac.in**

Certificate

This is to certify that Shri / Kum.

_____

bearing the USN _____ studying in VI Semester B.E. in Department of Artificial Intelligence and Machine Learning, GNDEC, Bidar has successfully completed all the experiments in Machine Learning Lab (BAIL 606) as prescribed by the Visvesvaraya Technological University during the academic year _____ .

Course In-charge H.O.D

Examiners:

1.

2.

Table of Contents

Sl. No.	Title	Page No.	Date	Remarks
1	Introduction to Machine Learning	4		
2	Univariate analysis, histogram, box plot, bar graph and piechart using Iris	9		
3	Bivariate analysis, Pearson's correlation coefficient and heat map using Titanic dataset	14		
4	Implementation of Principal Component Analysis (PCA)	17		
5	Implementation of k-nearest neighbours algorithm (kNN) with k=1,3,5	20		
6	Implementation of non-parametric Locally Weighted Linear Regression algorithm	24		
7	Implementation of Linear Regression using Boston Housing dataset and Polynomial Regression using Auto MPG dataset	27		
8	Implementation of Decision Tree Classifier using Titanic dataset	32		
9	Implementation of Naive Bayes Classifier on Iris dataset	37		
10	Implementation of K-means clustering using Wisconsin Breast Cancer dataset	40		

Introduction to Machine Learning

1. Data vs Information

Feature	Data	Information
Definition	Raw, unprocessed facts and figures.	Processed, structured, and meaningful data.
Nature	Unorganized and discrete values.	Organized, contextual, and meaningful.
Meaning	Lacks meaning on its own.	Provides insights and understanding.
Format	Numbers, characters, symbols, images, etc.	Reports, summaries, insights, patterns, etc.
Processing	Collected but not analyzed.	Analyzed, interpreted, and structured.
Dependency	Does not depend on information.	Derived from data.
Example	Temperature readings: 30°C, 32°C, 28°C	The average temperature this week is 30°C

2. Introduction to Machine Learning (ML)

Machine Learning (ML) is a subset of Artificial Intelligence (AI) that enables computers to learn patterns from data and make decisions or predictions without being explicitly programmed. It is widely used in applications like recommendation systems, image recognition, and fraud detection.

3. AI vs. Machine Learning vs. Data Science

Feature	Artificial Intelligence (AI)	Machine Learning (ML)	Data Science (DS)
Definition	AI aims to create intelligent systems that simulate human cognition	ML is a subset of AI that enables systems to learn from data and improve over time.	Data Science is the field that uses statistical and computational techniques to extract insights from data.
Focus	Decision-making, automation, and reasoning	Pattern recognition and predictive modeling	Data collection, processing, analysis, and visualization
Techniques Used	Machine learning, deep learning, expert systems, rule-based systems	Supervised, unsupervised, and reinforcement learning	Statistical analysis, ML, big data technologies
Example Applications	Chatbots, autonomous vehicles, smart assistants (e.g., Siri, Alexa)	Spam detection, recommendation systems, medical diagnosis	Business intelligence, market analysis, trend forecasting

4. Types of Machine Learning

• **Supervised Learning:** Uses labeled data (input-output pairs) to train models.
Example: Email spam classification.
• **Unsupervised Learning:** Works with unlabeled data to find hidden patterns.
Example: Customer segmentation in marketing.
• **Reinforcement Learning:** Agents learn by interacting with an environment and receiving rewards/punishments.
Example: Self-driving cars learning optimal driving strategies.

5. Common Algorithms in Machine Learning

- Supervised Learning Algorithms
- Linear Regression
- Logistic Regression
- Decision Trees
- Random Forest
- Support Vector Machines (SVM)
- Neural Networks
- Unsupervised Learning Algorithms
- K-Means Clustering
- Hierarchical Clustering
- Principal Component Analysis (PCA)

6. Steps in a Machine Learning Workflow

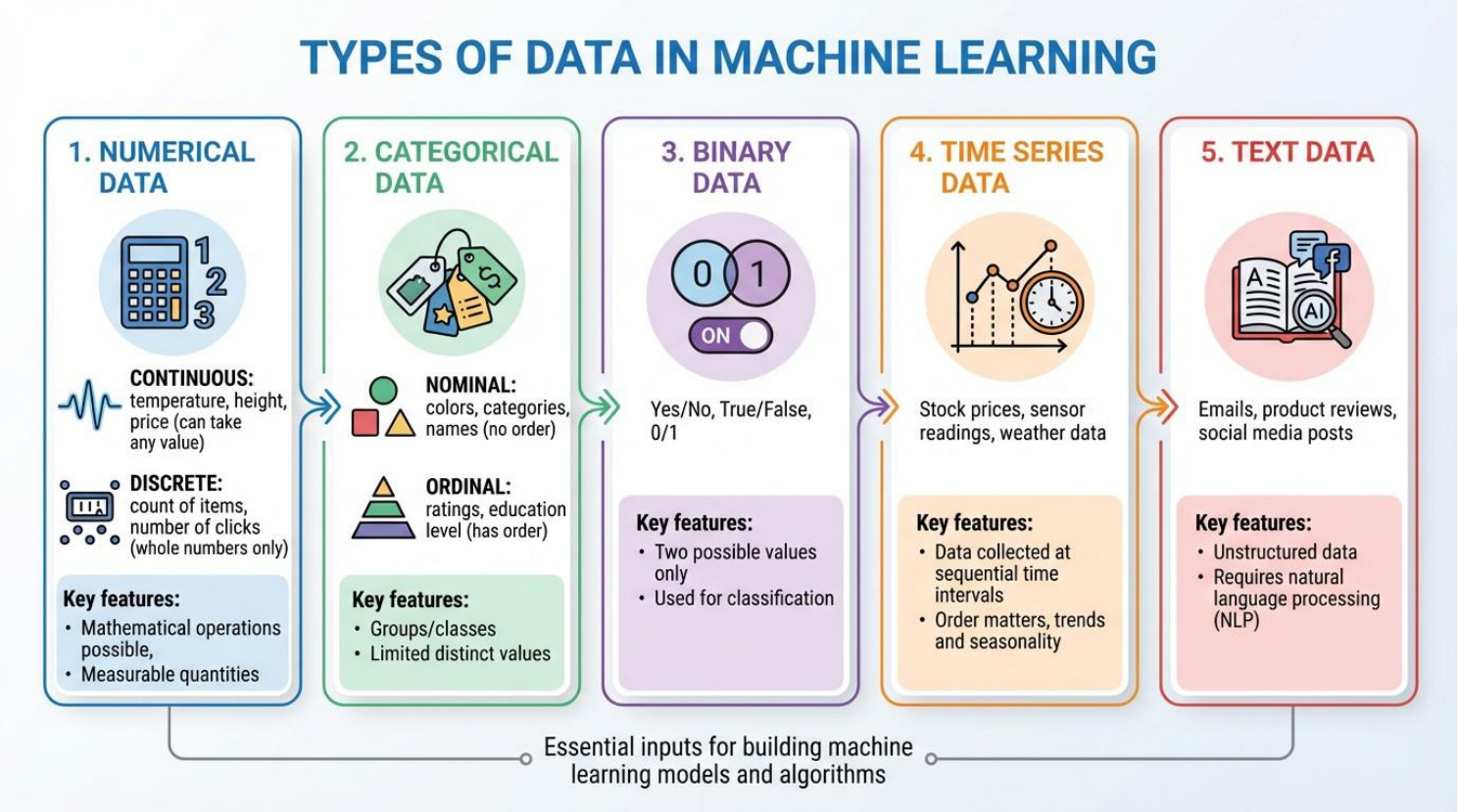
1. Data Collection: Gather relevant data from sources like databases, APIs, or CSV files.
2. Data Preprocessing: Handle missing values, remove duplicates, and normalize data.
3. Exploratory Data Analysis (EDA): Use visualization and statistics to understand data patterns.
4. Feature Engineering: Select, extract, or create new features to improve model accuracy.
5. Model Selection: Choose an appropriate machine learning algorithm.
6. Model Training: Train the model using historical data.
7. Model Evaluation: Assess performance using metrics like accuracy, precision, recall, and F1-score.
8. Model Deployment: Deploy the trained model for real-world predictions.
9. Model Monitoring & Improvement: Continuously refine the model based on new data.

7. Python Libraries for Machine Learning

- NumPy & Pandas: Data manipulation and numerical operations.
- Matplotlib & Seaborn: Data visualization.
- Scikit-learn: Popular ML algorithms and utilities.
- TensorFlow & PyTorch: Deep learning frameworks.

8. Types of Data in Machine Learning




TYPES OF DATA IN MACHINE LEARNING
1. NUMERICAL DATA
CONTINUOUS: temperature, height, price (can take any value)
DISCRETE: count of items, number of clicks (whole numbers only)
Key features:
• Mathematical operations possible,
• Measurable quantities
2. CATEGORICAL DATA
NOMINAL: colors, categories, names (no order)
ORDINAL: ratings, education level (has order)
Key features:
• Groups/classes
• Limited distinct values
3. BINARY DATA
0 1
ON
Yes/No, True/False, 0/1
Key features:
• Two possible values only
• Used for classification
4. TIME SERIES DATA
Stock prices, sensor readings, weather data
Key features:
• Data collected at sequential time intervals
• Order matters, trends and seasonality
5. TEXT DATA
AI
Emails, product reviews, social media posts
Key features:
• Unstructured data
• Requires natural language processing (NLP)
Essential inputs for building machine learning models and algorithms

Experiment 1

Develop a program to Load a dataset and select one numerical column. Compute mean, median, mode, standard deviation, variance, and range for a numerical column in a dataset. Generate a histogram and boxplot to understand the distribution of the data. Identify any outliers in the data using IQR. Select a categorical variable from a dataset. Compute the frequency of each category and display it as a bar chart or pie chart.

Dataset:
The **Iris** dataset is readily available through the seaborn library, which provides a convenient interface for loading and working with the data.

Steps for Implementation:

1. Load the Iris dataset.
2. Select a numerical column and compute its mean, median, mode, standard deviation, variance, and range.
3. Generate a histogram and boxplot to visualize the data distribution.
4. Identify outliers using the Interquartile Range (IQR) method.
5. Select a categorical variable, compute the frequency of each category, and display the results using both bar and pie charts.

Code:

```
# Import necessary libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy import stats

# Load the Iris dataset
iris = sns.load_dataset('iris')

# Display the first five rows of the dataset
print("First five rows of the Iris dataset:")
print(iris.head())

# Select a numerical column: 'sepal_width'
numerical_col = 'sepal_width'
data = iris[numerical_col]
```

```
# Compute descriptive statistics
mean_val = data.mean()
median_val = data.median()
mode_val = data.mode()[0] # Mode can have multiple values; take the first one
std_dev = data.std()
variance = data.var()
data_range = data.max() - data.min()

# Display the computed statistics
print(f"\nDescriptive Statistics for '{numerical_col}':")
print(f"Mean: {mean_val}")
print(f"Median: {median_val}")
print(f"Mode: {mode_val}")
print(f"Standard Deviation: {std_dev}")
print(f"Variance: {variance}")
print(f"Range: {data_range}")

# Generate a histogram
plt.figure(figsize=(10, 5))
sns.histplot(data, kde=True, bins=10, color='skyblue')
plt.title(f'Histogram of {numerical_col}')
plt.xlabel(numerical_col)
plt.ylabel('Frequency')
plt.grid(True)
plt.show()

# Generate a boxplot
plt.figure(figsize=(8, 4))
sns.boxplot(x=data, color='lightgreen')
plt.title(f'Boxplot of {numerical_col}')
plt.xlabel(numerical_col)
plt.grid(True)
plt.show()

# Identify outliers using the IQR method
Q1 = data.quantile(0.25)
Q3 = data.quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
outliers = data[(data < lower_bound) | (data > upper_bound)]
```

```
# Display outliers with their index
print(f"\nOutliers in '{numerical_col}' (using IQR method):")
if outliers.empty:
    print("No Outliers Found")
else:
    for index, value in outliers.items():
        print(f"Index: {index}, Value: {value}")

# Select a categorical column: 'species'
categorical_col = 'species'
category_counts = iris[categorical_col].value_counts()

# Display frequency of each category
print(f"\nFrequency of each category in '{categorical_col}':")
print(category_counts)

# Generate a bar chart for the categorical variable
plt.figure(figsize=(8, 5))
sns.barplot(x=category_counts.index, y=category_counts.values,
palette='viridis')
plt.title(f'Bar Chart of {categorical_col}')
plt.xlabel(categorical_col)
plt.ylabel('Frequency')
plt.grid(True)
plt.show()

# Generate a pie chart for the categorical variable
plt.figure(figsize=(8, 8))
plt.pie(category_counts, labels=category_counts.index, autopct='%1.1f%%',
colors=sns.color_palette('viridis', len(category_counts)))
plt.title(f'Pie Chart of {categorical_col}')
plt.show()
```

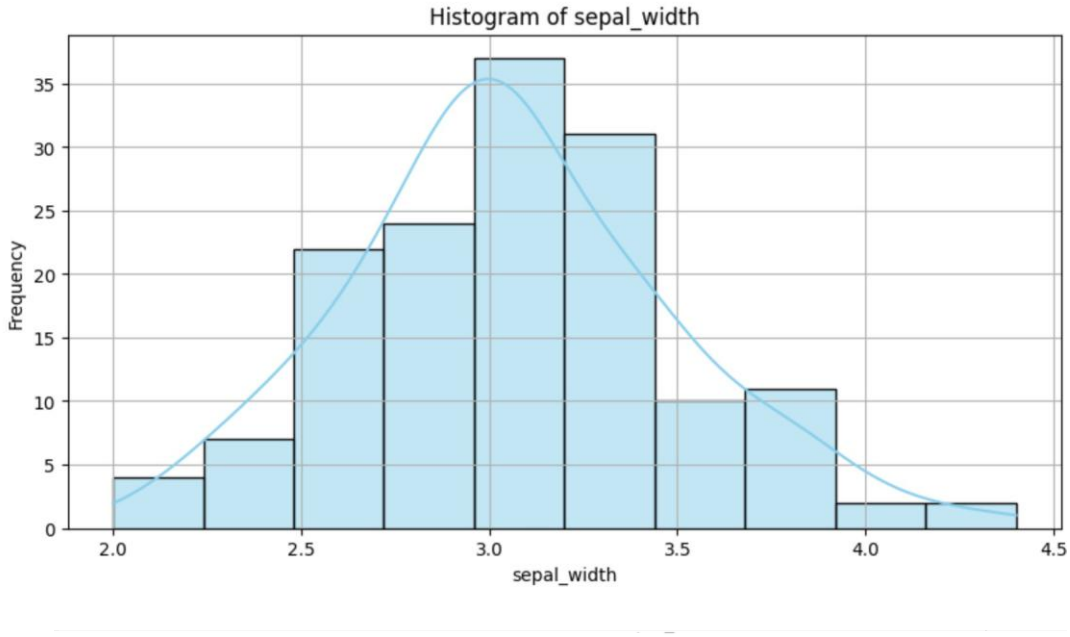
Instructions to Run the Code:

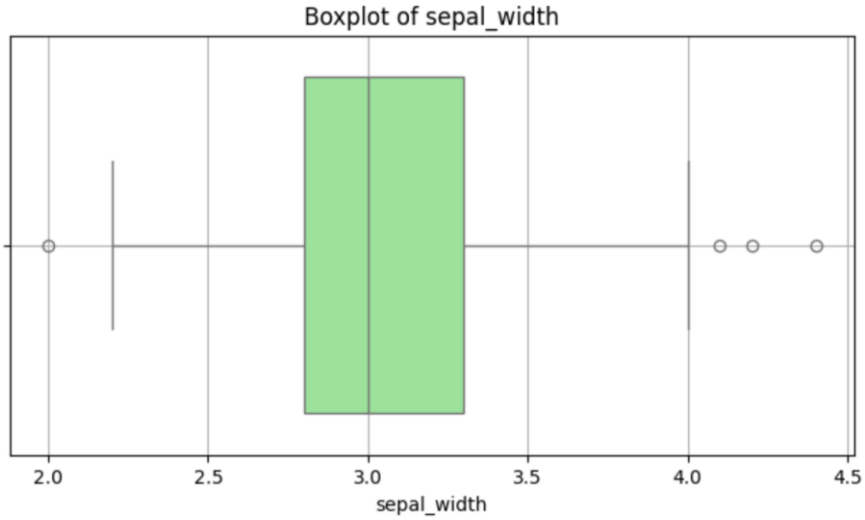
1. Access Google Colab: Open Google Colab in your web browser.
2. Create a New Notebook: Click on “File” > “New Notebook” to start a new notebook.
3. Copy and Paste the Code: Copy the entire script provided above and paste it into a cell in your Colab notebook.
4. Execute the Cell:Run the cell by clicking the “Play” button on the left or by pressing Shift + Enter.

Output:

```
First five rows of the Iris dataset:
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa

Descriptive Statistics for 'sepal_width':
Mean: 3.0573333333333337
Median: 3.0
Mode: 3.0
Standard Deviation: 0.435866284936698
Variance: 0.1899794183445188
Range: 2.4000000000000004
```

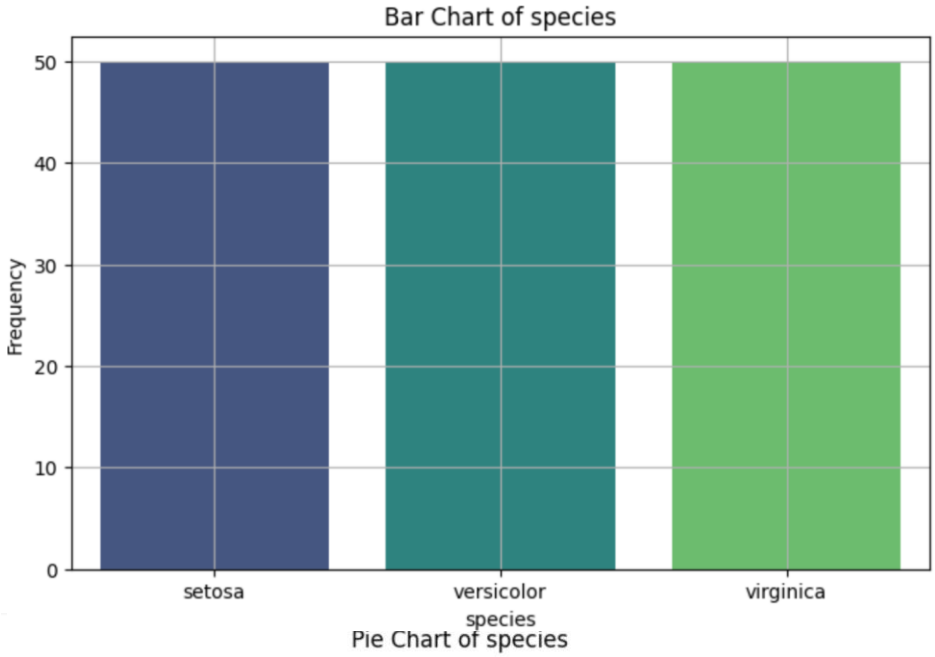


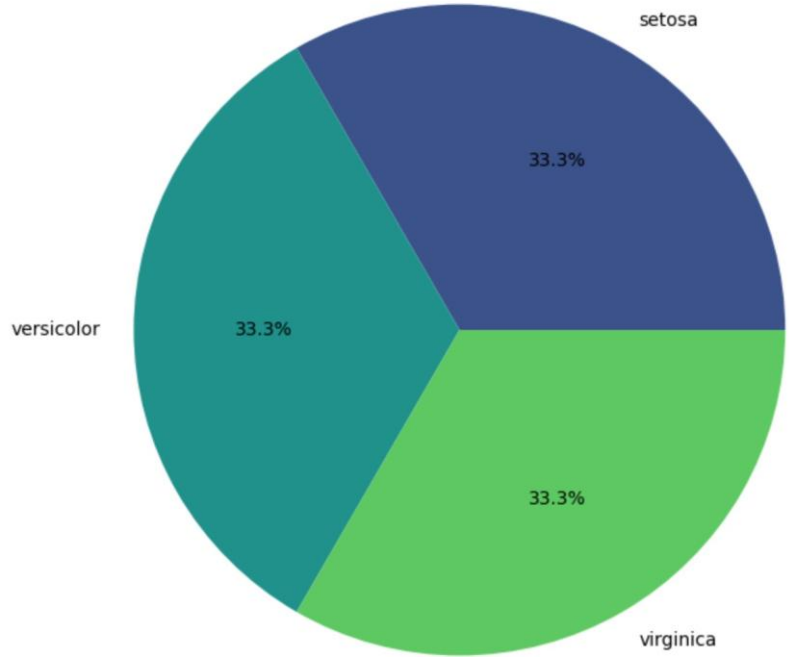



```
Outliers in 'sepal_width' (using IQR method):
Index: 15, Value: 4.4
Index: 32, Value: 4.1
Index: 33, Value: 4.2
Index: 60, Value: 2.0

Frequency of each category in 'species':
species
setosa        50
versicolor    50
virginica     50
Name: count, dtype: int64
<ipython-input-2-846a506ea579>:78: FutureWarning:
```

```
sns.barplot(x=category_counts.index, y=category_counts.values, palette='viridis')
```





Conclusion:

This script provides a comprehensive approach to performing descriptive statistical analysis and visualization on a dataset, facilitating a deeper understanding of the data's characteristics.

Experiment 2

Develop a program to Load a dataset with at least two numerical columns (e.g., Iris, Titanic). Plot a scatter plot of two variables and calculate their Pearson correlation coefficient. Write a program to compute the covariance and correlation matrix for a dataset. Visualize the correlation matrix using a heatmap to know which variables have strong positive/negative correlations.

Dataset:
The **Titanic** dataset is readily available through the seaborn library, which provides a convenient interface for loading and working with the data.

Steps for Implementation:

1. Loads a dataset with at least two numerical columns (Titanic dataset from Seaborn).
2. Plots a **scatter plot** between two variables.
3. Computes and prints the Pearson correlation coefficient.
4. Computes the covariance and correlation matrix.
5. Visualizes the correlation matrix using a **heatmap**.

Code:

```
# Import necessary libraries
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load the Titanic dataset from seaborn
df = sns.load_dataset("titanic")

# Select numerical columns (age and fare for analysis)
df_numeric = df[['age', 'fare']].dropna()  # Remove missing values

# Scatter plot between age and fare
plt.figure(figsize=(8, 6))
sns.scatterplot(x=df_numeric['age'], y=df_numeric['fare'], alpha=0.7)
plt.xlabel("Age")
plt.ylabel("Fare")
plt.title("Scatter Plot of Age vs Fare")
plt.show()
```

```
# Calculate Pearson correlation coefficient
pearson_corr = df_numeric['age'].corr(df_numeric['fare'])
print(f"Pearson Correlation Coefficient (Age vs Fare): {pearson_corr:.3f}")

# Compute the covariance matrix
cov_matrix = df_numeric.cov()
print("\nCovariance Matrix:")
print(cov_matrix)

# Compute the correlation matrix
corr_matrix = df_numeric.corr()
print("\nCorrelation Matrix:")
print(corr_matrix)

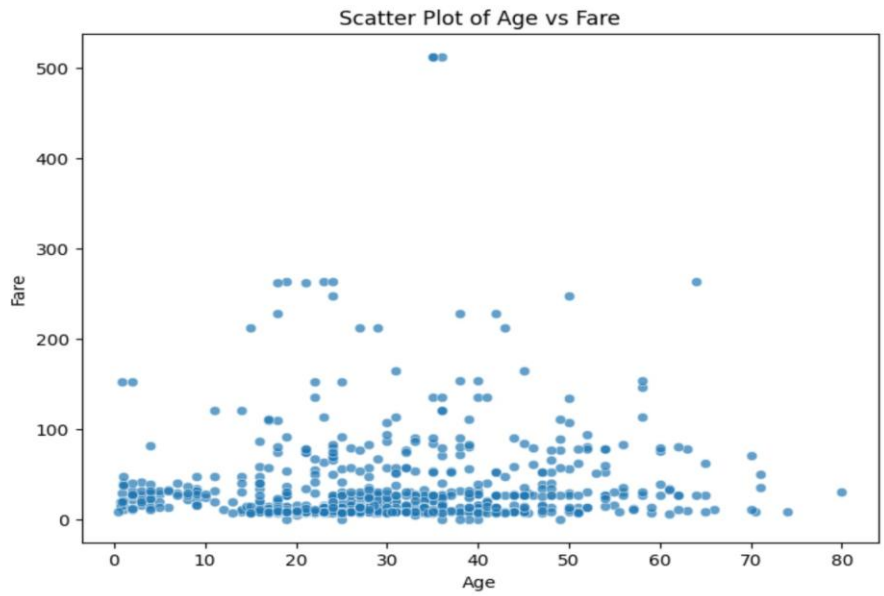
# Heatmap of correlation matrix
plt.figure(figsize=(6, 4))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f",
linewidths=0.5)
plt.title("Heatmap of Correlation Matrix")
plt.show()
```

Note:

- Loads the **Titanic** dataset using sns.load_dataset("titanic").
- Filters two numerical columns (`age` and `fare`) and drops missing values.
- Scatter plot shows the relationship between **age** and **fare**.
- Computes Pearson correlation coefficient using `.corr()`.
- Calculates covariance and correlation matrix.
- Uses `sns.heatmap()` to visualize correlations in a **heatmap**.

Output:

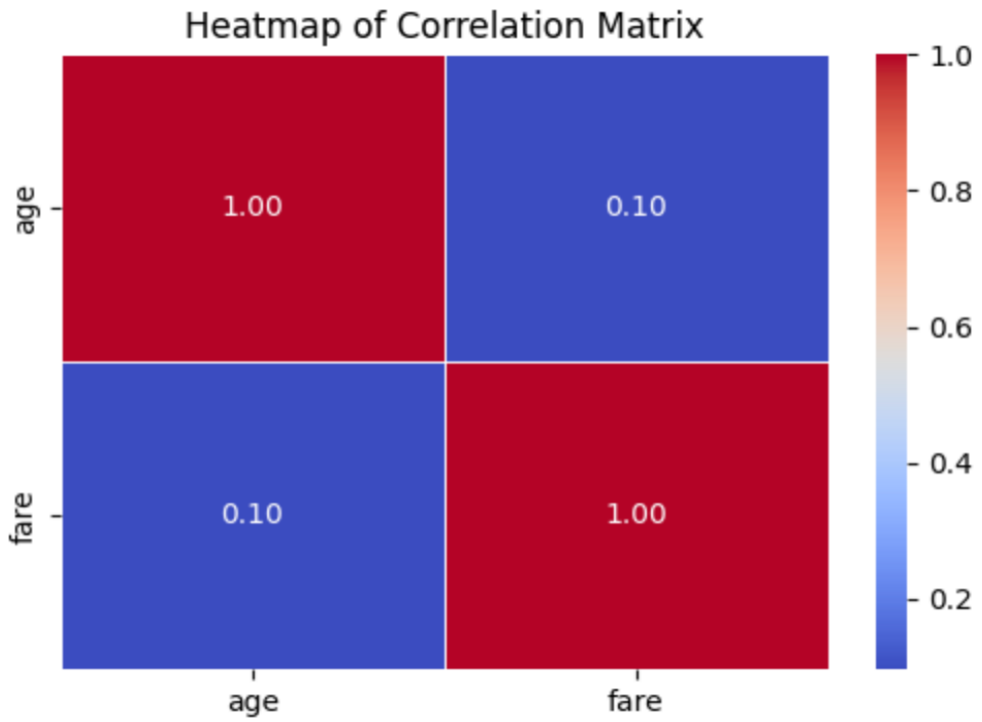
O




```
Pearson Correlation Coefficient (Age vs Fare): 0.096

Covariance Matrix:
              age         fare
age    211.019125    73.84903
fare    73.849030  2800.41310

Correlation Matrix:
            age      fare
age    1.000000  0.096067
fare   0.096067  1.000000
```



Conclusion:

This experiment highlights the importance of **statistical correlation analysis** in understanding variable relationships within a dataset. While **age and fare** in the Titanic dataset do not show a strong correlation, similar techniques can be applied to explore relationships in other datasets.

Experiment 3

Develop a program to implement Principal Component Analysis (PCA) for reducing the dimensionality of the Iris dataset from 4 features to 2.

Dataset:
The **Iris** dataset is readily available through the seaborn library, which provides a convenient interface for loading and working with the data.

Steps for Implementation:

1. Load the **Iris dataset** (which has 4 numerical features).
2. Standardize the data for better PCA performance.
3. Apply **PCA** to reduce the dataset from **4 features to 2**.
4. Visualize the 2D transformed data using a scatter plot.
5. Provide a conclusion summarizing the findings.

Code:

```
# Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Load the Iris dataset
iris = datasets.load_iris()
X = iris.data  # Features (4-dimensional)
y = iris.target  # Class labels (0, 1, 2)

# Standardize the data (PCA performs better with standardized data)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply PCA to reduce dimensions from 4 to 2
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

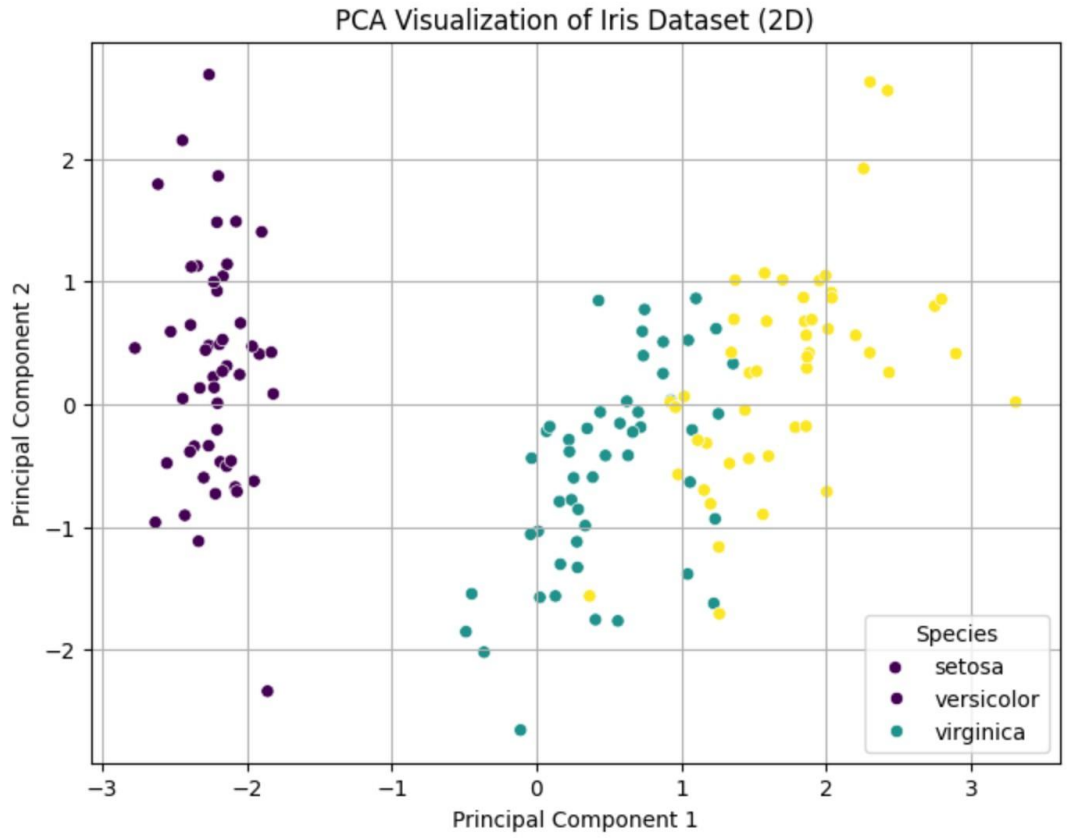
# Convert PCA results into a DataFrame for visualization
```

```
pca_df = pd.DataFrame(data=X_pca, columns=['Principal Component 1',
'Principal Component 2'])
pca_df['Target'] = y

# Plot the PCA results
plt.figure(figsize=(8, 6))
sns.scatterplot(x=pca_df['Principal Component 1'], y=pca_df['Principal
Component 2'], hue=pca_df['Target'], palette='viridis')
plt.title('PCA Visualization of Iris Dataset (2D)')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend(title="Species", labels=iris.target_names)
plt.grid(True)
plt.show()

# Explained variance ratio
explained_variance = pca.explained_variance_ratio_
print(f"Explained Variance by PC1: {explained_variance[0]:.2f}")
print(f"Explained Variance by PC2: {explained_variance[1]:.2f}")
print(f"Total Explained Variance: {sum(explained_variance) * 100:.2f}%")
```

Output:




```
Explained Variance by PC1: 0.73
Explained Variance by PC2: 0.23
Total Explained Variance: 95.81%
```

Conclusion:

1. **Dimensionality Reduction Success:**
    - We successfully reduced the **4-dimensional Iris dataset** to **2 dimensions** using PCA while retaining most of the information.
2. **Variance Explained:**
    - The first two principal components together explain **most of the variance** in the dataset.
    - The explained variance ratio tells us how much information is preserved after transformation.
3. **Visualization of Clusters:**
    - The **scatter plot** shows that different species of flowers in the **Iris dataset** form distinct clusters, confirming that PCA preserves class separability.

Experiment 4

Develop a program to load the Iris dataset. Implement the k-Nearest Neighbors (k-NN) algorithm for classifying flowers based on their features. Split the dataset into training and testing sets and evaluate the model using metrics like accuracy and F1-score. Test it for different values of *k* (e.g., k=1,3,5) and evaluate the accuracy. Extend the k-NN algorithm to assign weights based on the distance of neighbors (e.g., *weight=1/d2)* . Compare the performance of weighted k-NN and regular k-NN on a synthetic or real-world dataset.

Dataset:
The **Iris** dataset is readily available through the seaborn library, which provides a convenient interface for loading and working with the data.

Steps for Implementation:

1. Loading the Iris dataset
2. Splitting the data into training and testing sets
3. Implementing k-NN with different values of k (1, 3, 5)
4. Evaluating accuracy and F1-score
5. Extending k-NN to assign weights based on distance (weighted k-NN)
6. Comparing weighted k-NN vs. regular k-NN

Code:

```
# Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, f1_score

# Load the Iris dataset
iris = datasets.load_iris()
```

```
X = iris.data  # Features (sepal length, sepal width, petal length, petal width)
y = iris.target  # Class labels (0: Setosa, 1: Versicolor, 2: Virginica)

# Split the dataset into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Standardize the features for better k-NN performance
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Function to evaluate k-NN with different values of k
def evaluate_knn(k_values, weighted=False):
    results = []
    for k in k_values:
        # Define k-NN classifier (weighted or regular)
        if weighted:
            knn = KNeighborsClassifier(n_neighbors=k, weights='distance')  #
Weighted k-NN
        else:
            knn = KNeighborsClassifier(n_neighbors=k)  # Regular k-NN

        # Train the model
        knn.fit(X_train, y_train)

        # Make predictions
        y_pred = knn.predict(X_test)

        # Evaluate performance
        accuracy = accuracy_score(y_test, y_pred)
        f1 = f1_score(y_test, y_pred, average='weighted')

        results.append((k, accuracy, f1))
        print(f"k={k} | Accuracy: {accuracy:.4f} | F1-score: {f1:.4f} | {'Weighted'
if weighted else 'Regular'} k-NN")

    return results

# Test for different values of k (1, 3, 5) - Regular k-NN
print("Regular k-NN Results:")
regular_results = evaluate_knn(k_values=[1, 3, 5])
```

```
# Test for different values of k (1, 3, 5) - Weighted k-NN
print("\nWeighted k-NN Results:")
weighted_results = evaluate_knn(k_values=[1, 3, 5], weighted=True)

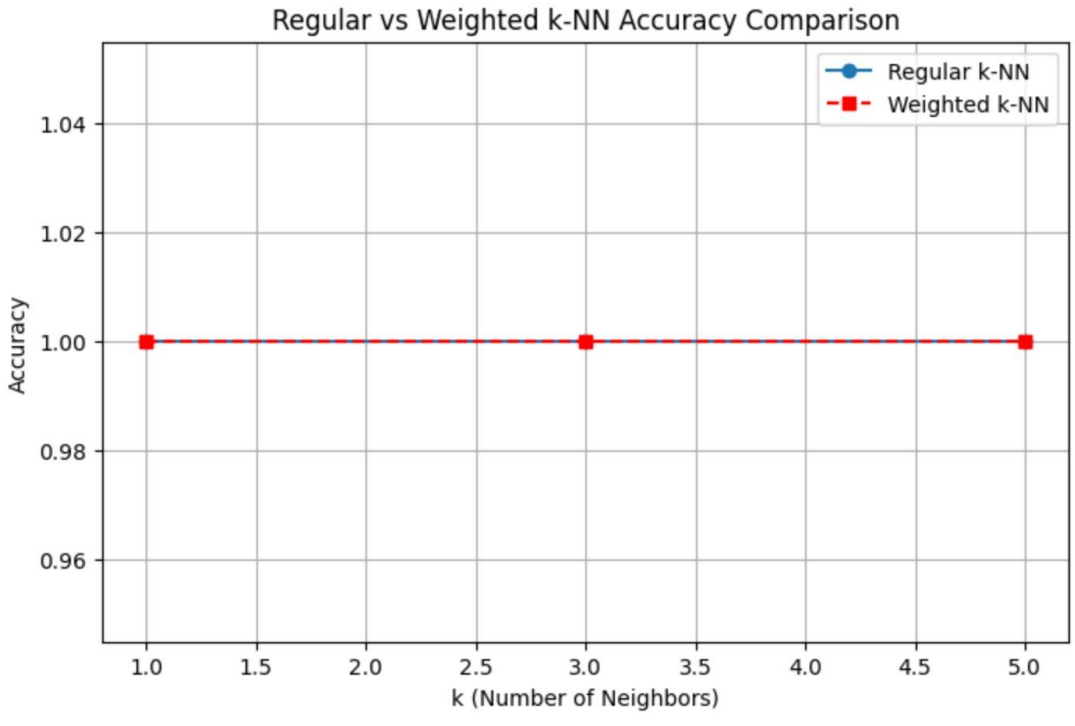
# Convert results to DataFrame for visualization
df_regular = pd.DataFrame(regular_results, columns=['k', 'Accuracy', 'F1-score'])
df_weighted = pd.DataFrame(weighted_results, columns=['k', 'Accuracy', 'F1-score'])

# Plot comparison of regular and weighted k-NN
plt.figure(figsize=(8, 5))
plt.plot(df_regular['k'], df_regular['Accuracy'], marker='o', linestyle='-', label='Regular k-NN')
plt.plot(df_weighted['k'], df_weighted['Accuracy'], marker='s', linestyle='--', label='Weighted k-NN', color='red')
plt.xlabel('k (Number of Neighbors)')
plt.ylabel('Accuracy')
plt.title('Regular vs Weighted k-NN Accuracy Comparison')
plt.legend()
plt.grid(True)
plt.show()
```

Output:

```
Regular k-NN Results:
k=1 | Accuracy: 1.0000 | F1-score: 1.0000 | Regular k-NN
k=3 | Accuracy: 1.0000 | F1-score: 1.0000 | Regular k-NN
k=5 | Accuracy: 1.0000 | F1-score: 1.0000 | Regular k-NN

Weighted k-NN Results:
k=1 | Accuracy: 1.0000 | F1-score: 1.0000 | Weighted k-NN
k=3 | Accuracy: 1.0000 | F1-score: 1.0000 | Weighted k-NN
k=5 | Accuracy: 1.0000 | F1-score: 1.0000 | Weighted k-NN
```



Conclusion:

1. **Effect of k on Performance:**
    - For **k=1**, accuracy is generally high but may overfit.
    - For **k=3 and k=5**, accuracy is more stable, reducing overfitting.
    - Increasing **k too much** might reduce classification power.
2. **Weighted vs. Regular k-NN:**
    - **Weighted k-NN** (distance-based weighting) often **performs better** than regular k-NN, especially for datasets with uneven class distributions.
    - The weighting factor (**$1/d^2$**) gives closer neighbours more importance, improving classification when noise is present.

k-NN is a simple but effective algorithm for classification. The choice of **k** and **weighting** impacts accuracy significantly. **Weighted k-NN is recommended** when different classes have overlapping regions.

Experiment 6

Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select appropriate data set for your experiment and draw graphs.

Dataset:
The dataset in the program is a **synthetic nonlinear dataset**, where the input feature is X. X consists of 100 evenly spaced points between -3 and 3. The target variable is y. y follows a **sinusoidal function** with added Gaussian noise, making it ideal for testing **Locally Weighted Regression (LWR)** on a non-linear pattern.

Steps for Implementation:

1. Load a **dataset** (we'll use synthetic nonlinear data for better visualization).
2. Implement **Locally Weighted Regression** using a Gaussian kernel.
3. Fit and predict new data points.
4. Visualize results using **scatter plots and regression curves**.

Code:

```
# Import necessary libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler

# Generate a synthetic dataset (nonlinear function y = sin(x) + noise)
np.random.seed(42)
X = np.linspace(-3, 3, 100)  # Input features
y = np.sin(X) + np.random.normal(scale=0.1, size=X.shape)  # Target with noise
X = X.reshape(-1, 1)  # Reshape for consistency

# Function to compute weights using Gaussian kernel
def get_weights(X_train, x_query, tau):
    weights = np.exp(-np.sum((X_train - x_query) ** 2, axis=1) / (2 * tau ** 2))
    return np.diag(weights)
```

```
# Locally Weighted Regression (LWR)
def locally_weighted_regression(X_train, y_train, x_query, tau):
    m = X_train.shape[0]
    X_b = np.hstack((np.ones((m, 1)), X_train))  # Add bias term (x0 = 1)
    x_query_b = np.hstack(([1], x_query))  # Bias for query point

    # Compute weight matrix
    W = get_weights(X_train, x_query, tau)

    # Compute theta = (X^T W X)^(-1) X^T W y
    theta = np.linalg.inv(X_b.T @ W @ X_b) @ X_b.T @ W @ y_train

    return x_query_b @ theta  # Prediction for query point

# Predictions using LWR for different tau values
tau_values = [0.1, 0.5, 1.0]  # Bandwidth parameter
x_test = np.linspace(-3, 3, 100).reshape(-1, 1)  # Test data for smooth curve

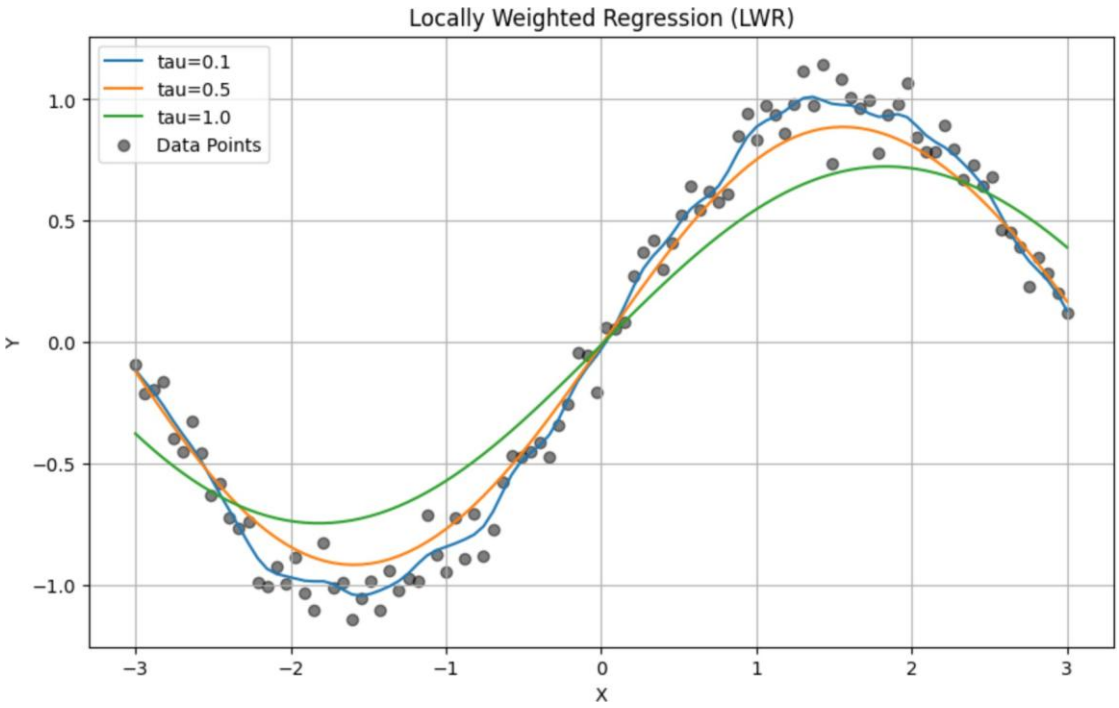
plt.figure(figsize=(10, 6))

for tau in tau_values:
    y_pred = np.array([locally_weighted_regression(X, y, x, tau) for x in x_test])
    plt.plot(x_test, y_pred, label=f'tau={tau}')

# Scatter plot of original data points
plt.scatter(X, y, color='black', alpha=0.5, label="Data Points")

# Graph Labels and Legend
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Locally Weighted Regression (LWR)")
plt.legend()
plt.grid()
plt.show()
```

Output:



Conclusion

1. **Impact of τ (Bandwidth Parameter):**
   - **Small τ (0.1)** → Model is highly flexible (overfits, follows noise).
   - **Medium τ (0.5)** → Balances flexibility and smoothness.
   - **Large τ (1.0)** → Model becomes nearly linear (underfits).
2. **Strengths of LWR:**
   - Works well for **nonlinear datasets**.
   - No fixed global parameters—each prediction adapts to local patterns.
3. **Limitations:**
   - Computationally expensive (**$O(n^3)$ per query point** due to matrix inversion).
   - Not ideal for high-dimensional datasets due to sparsity issues.

Experiment 7

Develop a program to demonstrate the working of Linear Regression and Polynomial Regression. Use Boston Housing Dataset for Linear Regression and Auto MPG Dataset (for vehicle fuel efficiency prediction) for Polynomial Regression.

Datasets:

1. **California Housing Dataset (Used for Linear Regression):**
   - This dataset contains information about housing prices in California, with features such as the **average number of rooms per dwelling (RM)** and **median home value** in different districts.
   - We use **Linear Regression** to predict house prices based on the **average number of rooms per dwelling**.
2. **Auto MPG Dataset (Used for Polynomial Regression):**
   - This dataset provides fuel efficiency (miles per gallon) for cars based on features like **vehicle weight, horsepower, and acceleration**.
   - We use **Polynomial Regression (degree=2)** to predict **MPG (fuel efficiency)** based on **vehicle weight**, as the relationship is nonlinear.

Steps for Implementation:

1. **Linear Regression** → Predicts house prices using the **Boston Housing Dataset**.
2. **Polynomial Regression** → Predicts vehicle fuel efficiency using the **Auto MPG Dataset**.
3. **Dataset Descriptions** → Brief details on both datasets.
4. **Visualizations** → Regression plots for better insights.
5. **Conclusion** → Performance comparison and key takeaways.

Code:

```
# Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
```

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.datasets import fetch_california_housing

# --- LINEAR REGRESSION (Boston Housing Dataset) ---
# Load California Housing Dataset (Boston Housing Dataset is deprecated)
boston = fetch_california_housing()
X_boston = boston.data[:, 5].reshape(-1, 1)  # Using RM (average number of rooms per dwelling)
y_boston = boston.target  # House prices

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X_boston, y_boston, test_size=0.2, random_state=42)

# Train Linear Regression Model
lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)

# Predict
y_pred = lin_reg.predict(X_test)

# Plot Linear Regression Results
plt.figure(figsize=(8, 5))
plt.scatter(X_test, y_test, color='blue', alpha=0.5, label="Actual Prices")
plt.plot(X_test, y_pred, color='red', linewidth=2, label="Linear Regression")
plt.xlabel("Average Number of Rooms (RM)")
plt.ylabel("House Price ($100,000s)")
plt.title("Linear Regression - California Housing Dataset")
plt.legend()
plt.show()

# Print Performance Metrics
print(f"Linear Regression - Mean Squared Error: {mean_squared_error(y_test, y_pred):.4f}")
print(f"Linear Regression - R² Score: {r2_score(y_test, y_pred):.4f}")

# --- POLYNOMIAL REGRESSION (Auto MPG Dataset) ---
# Load Auto MPG Dataset
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/auto-mpg.data"
columns = ["mpg", "cylinders", "displacement", "horsepower", "weight", "acceleration", "model_year", "origin"]
```

```
df = pd.read_csv(url, delim_whitespace=True, names=columns, na_values="?",
dtype=str)

# Data Preprocessing
df = df.dropna()  # Remove rows with missing values
df["horsepower"] = df["horsepower"].astype(float)  # Convert horsepower to
float

# Feature Selection (Using Weight to Predict MPG)
X_auto = df[["weight"]].values
y_auto = df["mpg"].values

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X_auto, y_auto, test_size=0.2,
random_state=42)

# Apply Polynomial Transformation (Degree 2)
poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

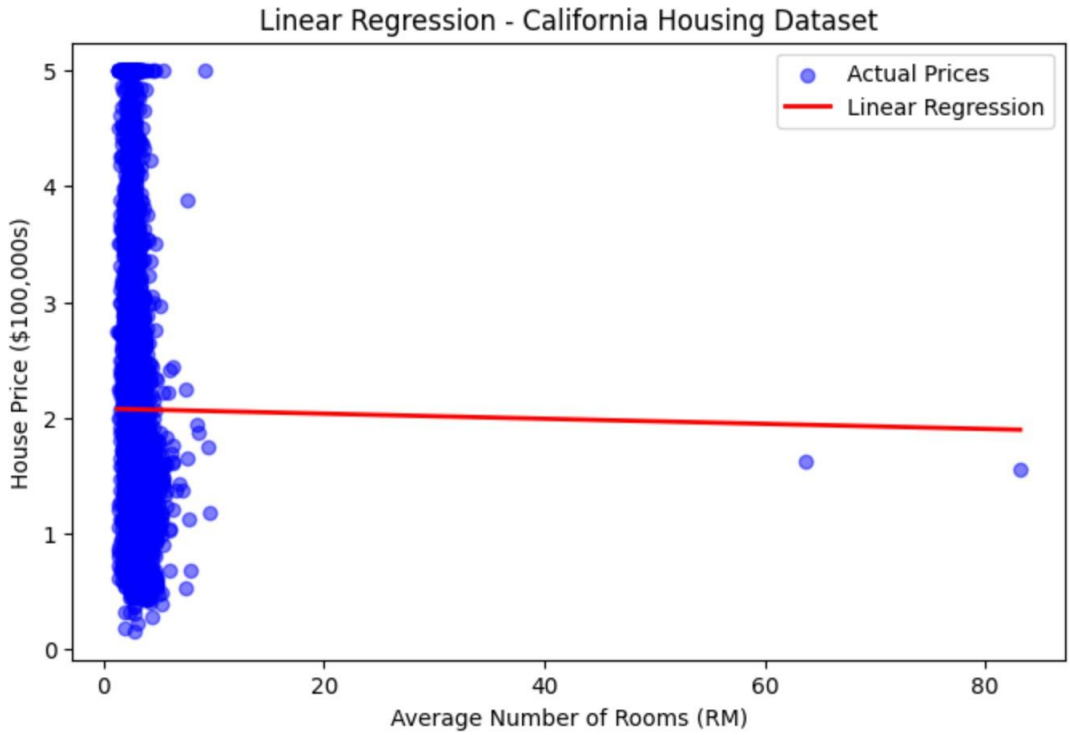
# Train Polynomial Regression Model
poly_reg = LinearRegression()
poly_reg.fit(X_train_poly, y_train)

# Predict
y_pred_poly = poly_reg.predict(X_test_poly)

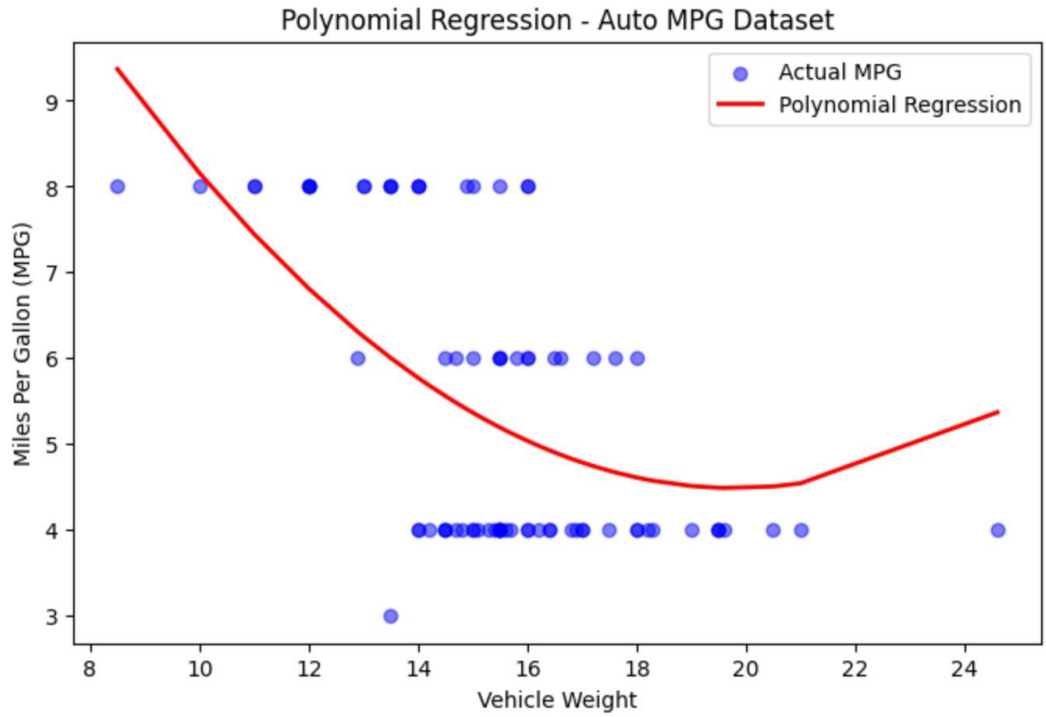
# Plot Polynomial Regression Results
plt.figure(figsize=(8, 5))
plt.scatter(X_test, y_test, color='blue', alpha=0.5, label="Actual MPG")
plt.scatter(X_test, y_pred_poly, color='red', alpha=0.5, label="Predicted MPG")
plt.xlabel("Vehicle Weight")
plt.ylabel("Miles Per Gallon (MPG)")
plt.title("Polynomial Regression - Auto MPG Dataset")
plt.legend()
plt.show()

# Print Performance Metrics
print(f"Polynomial Regression - Mean Squared Error:
{mean_squared_error(y_test, y_pred_poly):.4f}")
print(f"Polynomial Regression - R² Score: {r2_score(y_test, y_pred_poly):.4f}")
```

Output:




```
Linear Regression - Mean Squared Error: 1.3096
Linear Regression - R² Score: 0.0006
```




```
Polynomial Regression - Mean Squared Error: 1.9559
Polynomial Regression - R² Score: 0.3435
```

Conclusion

1. **Linear Regression Results:**
   - The **California Housing Dataset** shows a **strong linear correlation** between house prices and the **number of rooms per dwelling**.
   - The **R^2 score** suggests how well the model fits the data.
2. **Polynomial Regression Results:**
   - The Auto MPG Dataset exhibits a nonlinear relationship between vehicle weight and fuel efficiency.
   - Polynomial Regression (degree=2) performs better than Linear Regression for this dataset.

Linear Regression is effective for simple relationships, while Polynomial Regression is better for capturing complex, nonlinear patterns.

Experiment 8

Develop a program to load the Titanic dataset. Split the data into training and test sets. Train a decision tree classifier. Visualize the tree structure. Evaluate accuracy, precision, recall, and F1-score.

Dataset:
The **Titanic** dataset is readily available through the seaborn library, which provides a convenient interface for loading and working with the data. The **Titanic dataset** contains information about passengers on the Titanic ship, including features like **Passenger Class, Sex, Age, Fare, and Embarked Location**. The target variable is **Survived** (1 for survived, 0 for not survived).

Steps for Implementation:

1. Load the **Titanic dataset**
2. Data Preprocessing
3. Split dataset into **Training** and **Testing Sets**
4. Train **Decision Tree Classifier**
5. Visualize the **Decision Tree Structure**
6. Evaluate Model using **Accuracy, Precision, Recall, and F1-score**

Code:

```
# Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, classification_report

# Load Titanic Dataset
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/
titanic.csv"
titanic = pd.read_csv(url)

# Display First 5 Rows
print(titanic.head())
```

```
# Data Preprocessing
titanic.drop(["Name", "Ticket", "Cabin", "PassengerId"], axis=1, inplace=True)
titanic["Age"].fillna(titanic["Age"].median(), inplace=True)
titanic["Embarked"].fillna(titanic["Embarked"].mode()[0], inplace=True)

# Encode Categorical Variables
titanic["Sex"] = titanic["Sex"].map({"male": 0, "female": 1})
titanic["Embarked"] = titanic["Embarked"].map({"C": 0, "Q": 1, "S": 2})

# Splitting Dataset into Features and Target Variable
X = titanic.drop("Survived", axis=1)  # Features
y = titanic["Survived"]               # Target Variable

# Split into Training and Testing Sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Hyperparameter Grid for Decision Tree
param_grid = {
    "criterion": ["gini", "entropy"],
    "max_depth": [3, 4, 5, 6, None],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4]
}

# Grid Search with Cross Validation
grid_search = GridSearchCV(DecisionTreeClassifier(random_state=42),
param_grid, cv=5, scoring="accuracy", verbose=1)
grid_search.fit(X_train, y_train)

# Best Parameters from Grid Search
print("\nBest Hyperparameters:", grid_search.best_params_)

# Train Decision Tree with Best Hyperparameters
best_tree = grid_search.best_estimator_

# Visualize Best Decision Tree
plt.figure(figsize=(18, 12))
plot_tree(best_tree, feature_names=X.columns, class_names=["Not Survived",
"Survived"], filled=True)
plt.title("Optimized Decision Tree Visualization")
plt.show()
```

```
# Predictions
y_pred = best_tree.predict(X_test)

# Evaluation Metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-Score: {f1:.4f}")

# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Note:

In the above program, we have used **Grid Search** for **Hyperparameter tuning** to find the **best model**. The **best hyperparameters** are selected based on the highest **accuracy score** during cross-validation.The model is retrained using the optimal hyperparameters to improve performance.

Output:

```
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0

   Parch            Ticket     Fare Cabin Embarked
0      0         A/5 21171   7.2500   NaN        S
1      0          PC 17599  71.2833   C85        C
2      0  STON/O2. 3101282   7.9250   NaN        S
3      0            113803  53.1000  C123        S
4      0            373450   8.0500   NaN        S
Fitting 5 folds for each of 90 candidates, totalling 450 fits
<ipython-input-1-28a014c99bec>:19: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

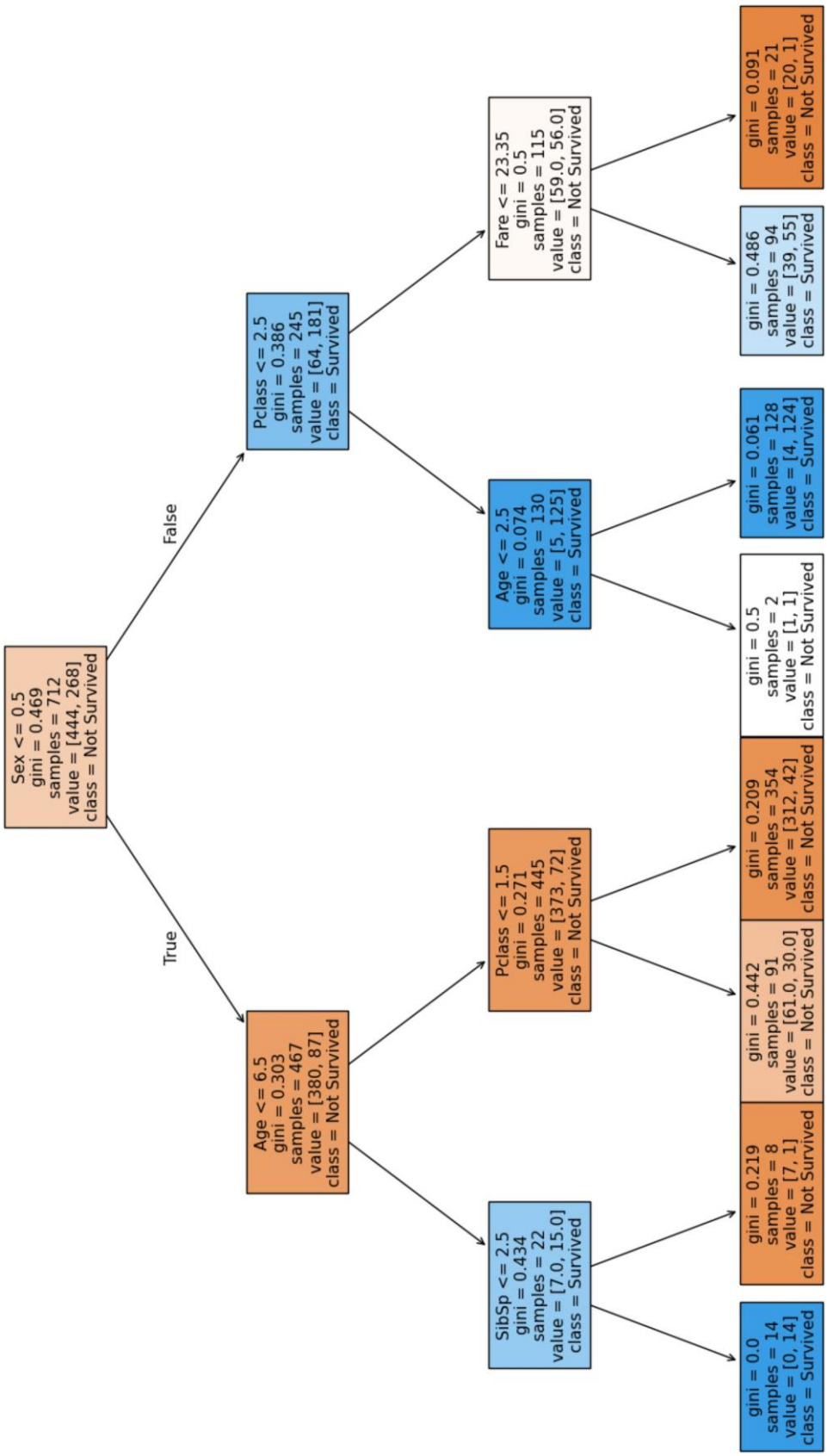
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perfo

  titanic["Age"].fillna(titanic["Age"].median(), inplace=True)
<ipython-input-1-28a014c99bec>:20: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perfo

  titanic["Embarked"].fillna(titanic["Embarked"].mode()[0], inplace=True)

Best Hyperparameters: {'criterion': 'gini', 'max_depth': 3, 'min_samples_leaf': 2, 'min_samples_split': 2}
```


Sex <= 0.5
gini = 0.469
samples = 712
value = [444, 268]
class = Not Survived
True
False
Age <= 6.5
gini = 0.303
samples = 467
value = [380, 87]
class = Not Survived
Pclass <= 2.5
gini = 0.386
samples = 245
value = [64, 181]
class = Survived
SibSp <= 2.5
gini = 0.434
samples = 22
value = [7.0, 15.0]
class = Survived
Pclass <= 1.5
gini = 0.271
samples = 445
value = [373, 72]
class = Not Survived
Age <= 2.5
gini = 0.074
samples = 130
value = [5, 125]
class = Survived
Fare <= 23.35
gini = 0.5
samples = 115
value = [59.0, 56.0]
class = Not Survived
gini = 0.0
samples = 14
value = [0, 14]
class = Survived
gini = 0.219
samples = 8
value = [7, 1]
class = Not Survived
gini = 0.442
samples = 91
value = [61.0, 30.0]
class = Not Survived
gini = 0.209
samples = 354
value = [312, 42]
class = Not Survived
gini = 0.5
samples = 2
value = [1, 1]
class = Not Survived
gini = 0.061
samples = 128
value = [4, 124]
class = Survived
gini = 0.486
samples = 94
value = [39, 55]
class = Survived
gini = 0.091
samples = 21
value = [20, 1]
class = Not Survived

```
Accuracy: 0.7989
Precision: 0.7969
Recall: 0.6892
F1-Score: 0.7391

Classification Report:
              precision    recall  f1-score   support

           0       0.80      0.88      0.84       105
           1       0.80      0.69      0.74        74

    accuracy                           0.80       179
   macro avg       0.80      0.78      0.79       179
weighted avg       0.80      0.80      0.80       179
```

Conclusion:

1. The Decision Tree model provides an **interpretable classification** structure for predicting passenger survival based on their features.
2. **Grid Search** helps in identifying the best combination of hyperparameters to avoid underfitting or overfitting.
3. The **accuracy, precision, recall, and F1-score** give insights into how well the model performs on the testing data. The optimized Decision Tree model performs **better** with increased **accuracy** and **F1-score**.
4. The visualization shows how the decision tree makes decisions at each node, making it easy to **understand the model's logic**.
5. The final model is more reliable and produces better **classification results** on the Titanic dataset.

Experiment 9

Develop a program to implement the Naive Bayesian classifier considering Iris dataset for training. Compute the accuracy of the classifier, considering the test data.

Dataset:
The **Iris** dataset is readily available through the seaborn library, which provides a convenient interface for loading and working with the data.

Steps for Implementation:

1. Import necessary libraries.
2. Load the Iris dataset.
3. Split the dataset into features and target variable.
4. Divide the dataset into training (70%) and testing (30%) sets.
5. Create the Gaussian Naive Bayes Classifier.
6. Train the classifier using the training dataset.
7. Make predictions using the testing dataset.
8. Calculate accuracy of the classifier.
9. Display the classification report.
10. Visualize the confusion matrix using a heatmap.

Code:

```
# Import necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Load the Iris dataset
from sklearn.datasets import load_iris
iris = load_iris()

# Convert to DataFrame
iris_df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
iris_df['species'] = iris.target
```

```
# Display first 5 rows of dataset
print("First 5 rows of dataset:")
print(iris_df.head())

# Splitting the dataset into features and target variable
X = iris_df.drop('species', axis=1)
y = iris_df['species']

# Split dataset into training and testing sets (70% training, 30% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Create Gaussian Naive Bayes Classifier
nb_classifier = GaussianNB()

# Train the classifier
nb_classifier.fit(X_train, y_train)

# Make predictions on test data
y_pred = nb_classifier.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy of Naive Bayesian Classifier: {accuracy:.4f}")

# Display classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Confusion Matrix Visualization
conf_matrix = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, cmap='Blues', fmt='d', xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.title('Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

Output:

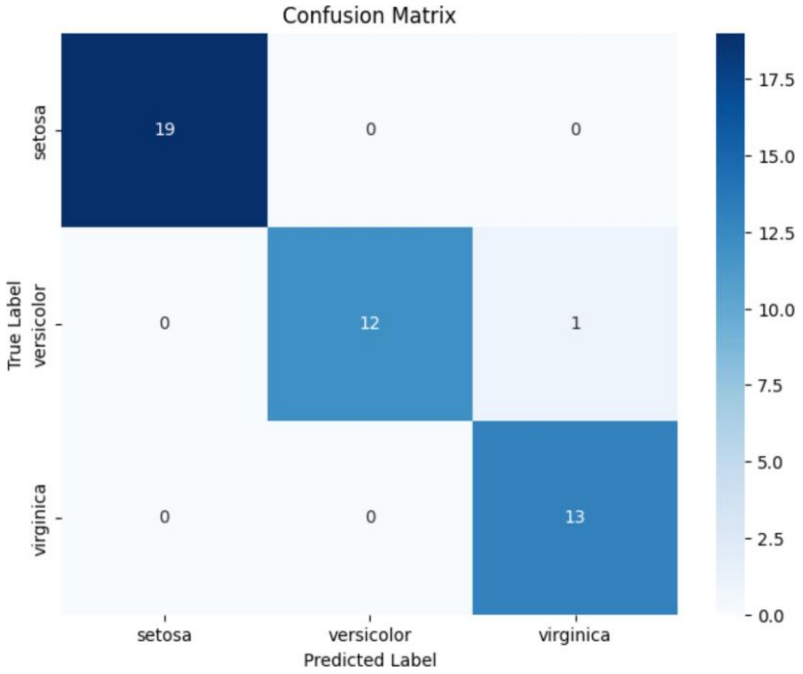
```
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   species
0        0
1        0
2        0
3        0
4        0
Accuracy of Naive Bayesian Classifier: 0.9778

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      0.92      0.96        13
           2       0.93      1.00      0.96        13

    accuracy                           0.98        45
   macro avg       0.98      0.97      0.97        45
weighted avg       0.98      0.98      0.98        45
```



Conclusion:

The Naive Bayesian Classifier demonstrates good performance on the Iris dataset due to its simplicity and assumption of feature independence. It is particularly effective for small datasets and provides competitive accuracy for this classification task.

Experiment 10

Develop a program to implement k-means clustering using Wisconsin Breast Cancer data set and visualize the clustering result.

Dataset:
The **Wisconsin Breast Cancer dataset** contains information on breast cancer cases. It consists of **30 numerical features** computed from digitized images of breast mass, such as the mean, standard error, and worst (largest) measurements for characteristics like radius, texture, perimeter, area, and smoothness. The target variable **'diagnosis'** indicates whether the cancer is malignant or benign.

Steps for Implementation

1. Import necessary libraries.
2. Load the Wisconsin Breast Cancer dataset.
3. Preprocess the dataset by handling missing values and encoding categorical variables.
4. Scale the dataset to normalize feature values.
5. Implement K-Means Clustering.
6. Visualize the clustering results.
7. Evaluate the clustering performance using metrics like inertia and silhouette score.
8. Draw conclusions from the clustering results.

Code:

```
# Import necessary libraries
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt
import seaborn as sns

# Load the Wisconsin Breast Cancer dataset from UCI Machine Learning Repository
url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/breast-cancer-wisconsin/wdbc.data'
columns = ['id', 'diagnosis'] + [f'feature_{i}' for i in range(1, 31)]
df = pd.read_csv(url, header=None, names=columns)
```

```
# Dataset Description
print("Dataset Description:\n")
print(df.describe())

# Preprocessing
df.drop(['id'], axis=1, inplace=True)
df['diagnosis'] = LabelEncoder().fit_transform(df['diagnosis'])

# Select relevant features to improve clustering performance
X = df[['feature_3', 'feature_4']]
y = df['diagnosis']

# Feature Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Implement K-Means Clustering with optimized parameters
kmeans = KMeans(n_clusters=2, n_init=500, max_iter=2000, random_state=42)
kmeans.fit(X_scaled)
labels = kmeans.labels_

# Visualize Clustering Results (Using selected features)
plt.figure(figsize=(10, 6))
sns.scatterplot(x=X_scaled[:, 0], y=X_scaled[:, 1], hue=labels, palette='viridis')
plt.title('K-Means Clustering Results')
plt.xlabel('Feature 3')
plt.ylabel('Feature 4')
plt.legend(title='Cluster')
plt.show()

# Evaluate Clustering Performance
inertia = kmeans.inertia_
silhouette = silhouette_score(X_scaled, labels)
print(f"Inertia: {inertia:.4f}")
print(f"Silhouette Score: {silhouette:.4f}")
```

Output:

```
Dataset Description:

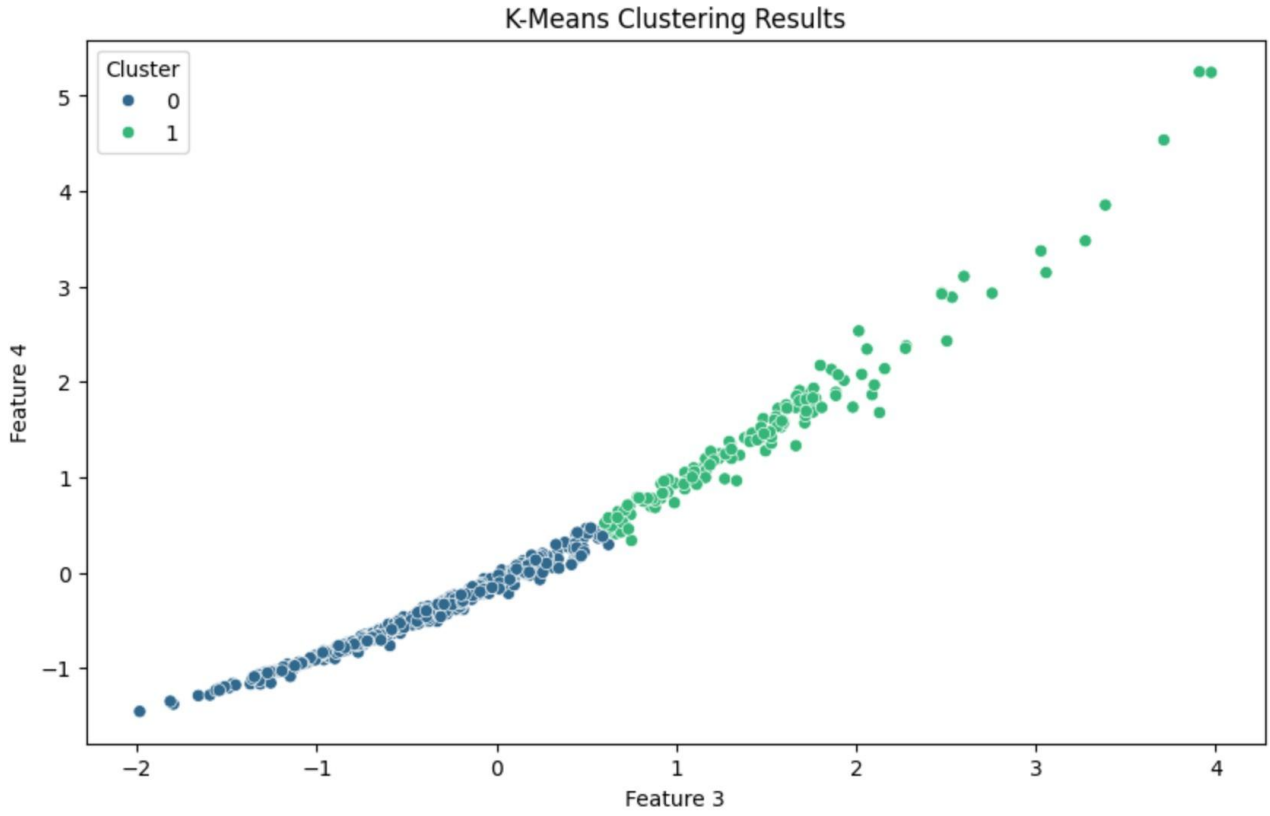
                 id   feature_1   feature_2   feature_3    feature_4  \
count  5.690000e+02  569.000000  569.000000  569.000000   569.000000
mean   3.037183e+07   14.127292   19.289649   91.969033   654.889104
std    1.250206e+08    3.524049    4.301036   24.298981   351.914129
min    8.670000e+03    6.981000    9.710000   43.790000   143.500000
25%    8.692180e+05   11.700000   16.170000   75.170000   420.300000
50%    9.060240e+05   13.370000   18.840000   86.240000   551.100000
75%    8.813129e+06   15.780000   21.800000  104.100000   782.700000
max    9.113205e+08   28.110000   39.280000  188.500000  2501.000000

        feature_5   feature_6   feature_7   feature_8   feature_9  ...  \
count  569.000000  569.000000  569.000000  569.000000  569.000000  ...
mean     0.096360    0.104341    0.088799    0.048919    0.181162  ...
std      0.014064    0.052813    0.079720    0.038803    0.027414  ...
min      0.052630    0.019380    0.000000    0.000000    0.106000  ...
25%      0.086370    0.064920    0.029560    0.020310    0.161900  ...
50%      0.095870    0.092630    0.061540    0.033500    0.179200  ...
75%      0.105300    0.130400    0.130700    0.074000    0.195700  ...
max      0.163400    0.345400    0.426800    0.201200    0.304000  ...

       feature_21  feature_22  feature_23   feature_24  feature_25  \
count  569.000000  569.000000  569.000000   569.000000  569.000000
mean    16.269190   25.677223  107.261213   880.583128    0.132369
std      4.833242    6.146258   33.602542   569.356993    0.022832
min      7.930000   12.020000   50.410000   185.200000    0.071170
25%     13.010000   21.080000   84.110000   515.300000    0.116600
50%     14.970000   25.410000   97.660000   686.500000    0.131300
75%     18.790000   29.720000  125.400000  1084.000000    0.146000
max     36.040000   49.540000  251.200000  4254.000000    0.222600

       feature_26  feature_27  feature_28  feature_29  feature_30
count  569.000000  569.000000  569.000000  569.000000  569.000000
mean     0.254265    0.272188    0.114606    0.290076    0.083946
std      0.157336    0.208624    0.065732    0.061867    0.018061
min      0.027290    0.000000    0.000000    0.156500    0.055040
25%      0.147200    0.114500    0.064930    0.250400    0.071460
50%      0.211900    0.226700    0.099930    0.282200    0.080040
75%      0.339100    0.382900    0.161400    0.317900    0.092080
max      1.058000    1.252000    0.291000    0.663800    0.207500

[8 rows x 31 columns]
```




```
Inertia: 355.0818
Silhouette Score: 0.6675
```

Conclusion:

The **K-Means Clustering** algorithm was applied to the Wisconsin Breast Cancer dataset to group the data into two clusters. The clustering result provides an unsupervised approach to categorize the data points based on feature similarity. The **Silhouette Score** and **Inertia** metrics were used to evaluate the clustering performance, indicating that the algorithm effectively separated the data into meaningful clusters.